Week 4 – Casino Revenue Forecasting, Model Evaluation and Observability

How to read this document. The brief's own text is unchanged. Under each point, a blue box says what we did, how, and the result, as of 13 November 2026. Numbers such as D28 are decisions in docs/week4-decision-log.md, which records what was decided, why and by whom.

The project in brief. We built a monthly forecasting and monitoring system for casino gaming win (what casinos keep from gambling) in two Nevada markets, the Las Vegas Strip and Reno, from the Gaming Control Board's own reports.

– **Data:** 234 Board reports, January 2015 to September 2026 (2020–21 left out for COVID), read two independent ways and cross-checked.

– **Models:** two simple baselines and seven models, tested over 43 months as if they had been running live, forecasting two months ahead.

– **Winner:** 'same month last year' for the Strip (5.75% average miss). Reno switched to Seasonal ARIMA + game mix (V2) after passing the promotion rule and Product Owner approval.

– **Running it:** MLflow records every experiment and the registered versions; a monthly monitoring run checks the data, scores forecasts, raises drift alerts, runs the Champion/Challenger check, and makes the next forecast with an 80% range; a Streamlit dashboard shows it all.

– **Latest (data to September 2026):** both markets Healthy. November 2026 forecast: Strip $784.3m (80% range $724m to $893m), Reno $64.4m ($58m to $73m).

– **Code and documents:** github.com/tezla-timemachine/casino-revenue-forecasting (public).

– **Technology:** Python 3.10 throughout. The key tools are:

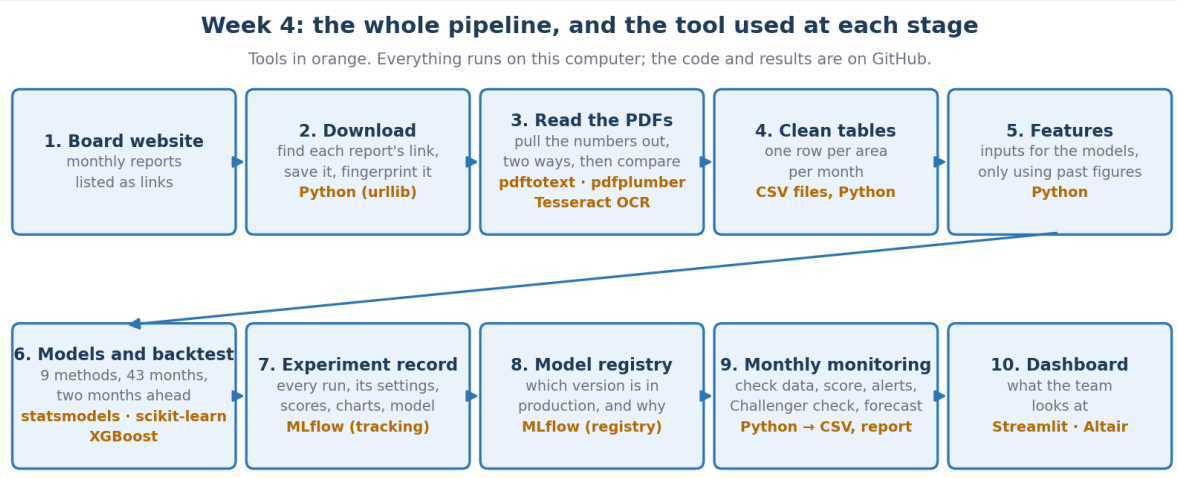
· pdftotext, pdfplumber and Tesseract to read the PDFs;

· statsmodels, scikit-learn and XGBoost for the models;

· MLflow to record experiments and model versions;

· Streamlit for the dashboard.

Each is explained where it's used. The diagram shows where each one fits.



Project Title

Casino Revenue Forecasting and Model Monitoring Platform

What we did: built it under this name. Everything is in one project folder and a public GitHub repository: scripts, clean data, model results, monitoring outputs, dashboard and documents.

Business Scenario

A casino industry analytics team receives monthly gaming revenue data from different Nevada casino markets.

What we did: we play that analytics team. Each month the Board publishes two reports, and we hold 234 of them: 117 two-page monthly releases (market totals) and 117 Monthly Revenue Reports of about 48 pages (game-by-game detail, table and machine counts, hold), January 2015 to September 2026.
How the files are fetched: two Python scripts (fetch_arr.py and fetch_board_reports.py):
– open the Board's web pages with Python's built-in web tool (urllib);
– find every report link on the page, and work out each report's month from the link text;
– compare that list with what's already saved, and download only what's missing, and only when asked;
– check each file is a real PDF, and record its SHA-256 fingerprint: a code calculated from the file's contents. If a file ever changed, its fingerprint would too.

Management does not only want to know what happened in the previous month. They also want to understand what revenue can be expected in the coming month, which forecasting model should be trusted, whether the model is still performing well, and when it should be retrained.

What we did: each question has a working answer that updates monthly.
– **Coming month:** a forecast two months ahead, with an 80% range.
– **Which model to trust:** nine methods tested over 43 months.
– **Still performing well:** monthly monitoring gives each market a status: Healthy, Investigate or Data issue.
– **When to retrain:** a written policy (D39), tested on our results.

The purpose of this project is to build a complete machine learning lifecycle around casino revenue forecasting.

What we did: covered the whole lifecycle: get the data → clean and check it → build features → compare models → track experiments in MLflow → register versions → run Champion against Challengers → monitor data quality, performance and drift → retrain by policy → dashboard → decisions and risks written down.

The focus should not only be on model training. The project should cover model comparison, experiment tracking, model registration, monitoring, drift detection, and retraining decisions.

What we did: all six are built and working:
– **Comparison:** nine methods, three measures, 43 months.
– **Tracking:** 20 MLflow runs.
– **Registration:** a registered model per market, with versions labelled production and challenger.
– **Monitoring and drift:** a monthly run with four kinds of alert.
– **Retraining:** a policy, decided from tests on our own results.

Dataset

Use the official Nevada Gaming Control Board revenue data.

What we did: used only the Board's own publications.

– The **monthly release** gives the total win for the state and 20 areas. This is the figure we forecast.

– The **Monthly Revenue Report** adds win by game (slots, blackjack, baccarat, craps, sports betting …), the number of venues, tables and slot machines, and hold (the share of money bet that casinos keep).

We first catalogued every layout the reports have used: six across the years (docs/week4-report-catalogue.md).

How the PDFs were turned into numbers is explained in the next box, with a diagram.

Monthly Revenue Release:

https://www.gaming.nv.gov/about-us/abbreviated-revenue-release-arr/

What we did, part 1: getting the files. scripts/fetch_arr.py opens this exact page each time and finds every report link. It works out each report's month from the link, lists any month missing or listed twice, and downloads only when asked.

It found two links both labelled 'Jan 2025'; one is really November 2024. So the script trusts the month in the link text and flags the mismatch (D7).

Part 2: reading the files. A PDF holds a page as text, pictures and positions. A program can pull the text out, but a table in a PDF is only words placed at positions on the page. The rows and columns have to be rebuilt.

One misread digit would quietly spoil every forecast. So each monthly release was read by two separate readers, and the results compared:

– **Reader A, pdftotext** (part of Poppler, a standard free PDF toolkit): copies the page as plain text, keeping its layout, so each table row stays on one line. Our rules then pick out each area's name and figures.

– **Reader B, pdfplumber** (a Python library): lists every word with its exact position on the page. The rows are rebuilt from those positions, using the same rules.

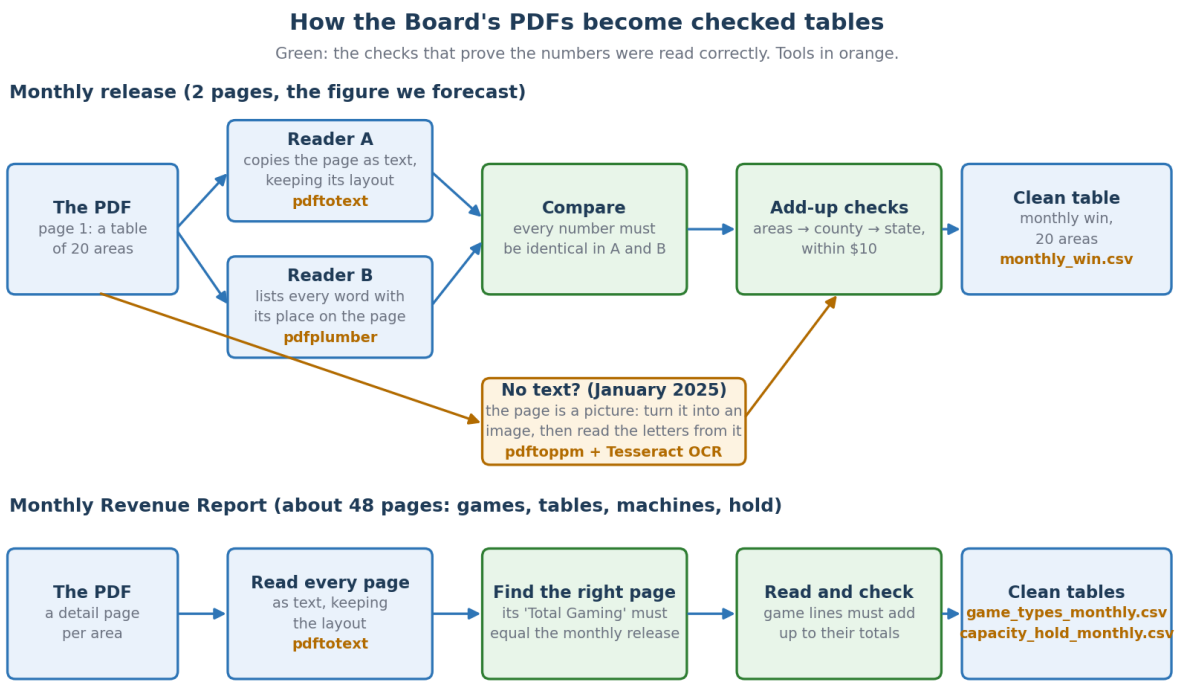
– **Compare:** every number must be identical in both. All are, for all 117 months (D10, D11).

– **Add-up checks:** each county's areas must add up to the county, and the counties to the state, within $10. Each figure is also checked against the same month in the next year's report.

– **Picture pages:** January 2025's table is a picture with no text inside it. pdftoppm turns the page into an image, then Tesseract reads it. Tesseract is free OCR software: it recognises letters and digits in an image. The result passed every check above.

– **The 48-page Monthly Revenue Report** is read with pdftotext. Each area's detail page is found by its 'Total Gaming' figure, which must equal that area's figure in the monthly release. Then the game lines must add up to their totals.

Code: scripts/extract_reports.py (the two readers and the OCR), build_table.py, build_game_types.py, extract_capacity_hold.py.



Select a few casino markets for the project, such as Las Vegas Strip, Downtown Las Vegas, Reno, or Laughlin.

What we did: chose the **Las Vegas Strip** (about 56% of state win) and **Reno** (about 5%) for models: the largest and steadiest markets (D14).

Downtown and Laughlin data is held and analysed in the exploratory analysis. Their models are not built yet (D29), a known gap.

Main Business Questions

The system should help answer the following questions.

- What revenue should we expect next month?

  What we did: each month, monitoring forecasts the month two ahead, because the Board publishes about four weeks after a month ends (D19). With data to September 2026, the November 2026 forecasts are:

  – **Strip: $784.3m**, 80% range $724m to $893m.

  – **Reno: $64.4m**, 80% range $58m to $73m.

  The range means 8 months in 10 should land inside. It comes from each model's own past misses, and held 86% (Strip) and 87% (Reno) of the time on the test (D53).

  It's calculated in scripts/ranges.py with numpy, Python's library for working with numbers.

- Which forecasting model performs best?

  What we did: tested nine methods on the same 43 months, March 2023 to September 2026.

– **Strip:** 'same month last year' had the lowest average miss: 5.75%, about $44m a month. A later bottom-up method that takes baccarat luck out did better (5.28%). It now runs as a Challenger.

– **Reno:** 'same month last year' and Seasonal ARIMA + game mix were nearly level (7.24% and 7.21%). The ARIMA model then won on recent months and was promoted.

Full table under Step 6.

- Is the current production model still reliable?

What we did: answered every month by monitoring. Latest run (data to September 2026): both markets are **Healthy**.

– **Strip:** 12-month average miss 5.39%.

– **Reno:** 6.76%.

Ten of the last twelve Strip results and nine of Reno's landed inside their 80% range.

- Has the behaviour of a casino market changed over time?

What we did: yes, and we measured how (docs/week4-exploratory-analysis.md):

– Markets reset 25–40% higher after COVID.

– Growth has stalled since 2023, except Reno (+8% in 2026).

– The Strip's December peak got stronger.

– Slot machines fell about 20% while each earns about twice as much.

– Strip forecast misses follow baccarat luck (correlation 0.75).

Drift alerts now watch for further change every month (D37).

- Is the latest prediction error acceptable?

What we did: defined 'acceptable' from each model's own history (D37). A month's miss is a bad month if it's in the model's worst 10%: over 12.4% for the Strip's model, over 13.8% for Reno's.

– One bad month alone is accepted.

– Two in three months, or a 12-month average more than 1.5 points above normal, turns the status to **Investigate**.

September 2026's misses were +0.8% (Strip) and −3.7% (Reno): acceptable.

- When should the model be retrained?

What we did: a two-layer policy (D39).

– **Routine:** models that learn settings re-learn them every month on all data published so far.

– **Full review:** yearly, and also when a slow alert fires, tables or machines change by more than 5% for a real reason, or a fast alert has a lasting cause.

Tested on our results: monthly retraining took the Strip's ARIMA model from 7.67% (never retrained) to 6.14%.

- Should a new model replace the current production model?

What we did: an agreed promotion rule, checked every month (D32). A Challenger must, at three monthly checks in a row:

– be at least 0.5 points better over the last 12 months;

– be no more than 0.25 points worse over the whole test;

– not have worse big misses (RMSE) or a larger lean (bias).

The Product Owner then approves (D34). Reno met it and switched to V2. Neither Strip Challenger meets it now.

Step 1: Prepare the Data

- Download the historical revenue reports from the Nevada Gaming Control Board.

  What we did: downloaded 234 reports: 117 monthly releases (23 MB) and 117 Monthly Revenue Reports (82 MB), January 2015 to December 2019 and January 2022 to September 2026.
  Each file is checked to be a real PDF, and its size and fingerprint go into a list (manifest). You approved each batch. 2020 and 2021 aren't held, because COVID closures make them misleading (D23).

- Extract the available information from the reports and convert it into a structured dataset that can be used for forecasting.

  What we did: every monthly release was read by two separate methods (pdftotext and pdfplumber), and every number compared. All agree. This is explained, with a diagram, under the dataset link above. The January 2025 table is a picture, so it was read with OCR and then checked. Each area's figures must add up to its county, and counties to the state, within $10.
  The result is four clean tables, saved as CSV files: plain tables that Excel, Python or any tool can open.
  – **Monthly win:** 2,340 rows, 20 areas × 117 months.
  – **Win by game type:** 9,551 rows.
  – **Capacity and hold:** 585 rows.
  – **Model inputs:** 86 rows.

- The student should first understand what information is actually available in the source reports before creating the final dataset.

  What we did: read the September 2026 release line by line before building anything, then grouped all 234 reports by layout. We found:
  – six layouts;
  – the full list of fields, and which we use (docs/week4-report-fields.md);
  – that page 2 (tax collected) covers the whole state only, so it isn't used;
  – that the big report also gives venue, table and machine counts and hold. These were missed at first and added after review (D30).

- Keep the original source information separate from any fields created during data preparation.

  What we did: partly.
  – **Kept separate:** the capacity table holds only source fields, and in the model-input table every created column starts with derived_.
  – **Gap:** in the main monthly-win table, a few created fields (grouping, revised flag) sit beside the source figures. Splitting it into two tables is a known small gap.

- After extracting the source data, create any additional features that are required for forecasting.

  What we did: built 14 inputs for the Strip and Reno, using Python scripts (build_features.py and build_calendar.py) that read the clean tables. Each uses only figures published at least two months before the month being forecast, so no input could 'see the future' (D18, D19).

- These may include historical lag values, previous period values, moving averages, percentage changes, seasonal indicators, or other useful time series features.

  What we did: covered each type the brief lists:
  – **Lag:** same month last year.
  – **Previous period:** newest published month.
  – **Moving averages:** 3 and 12 months.
  – **Percentage changes:** on a year ago and on the month before.
  – **Seasonal indicator:** month of the year.
  – **Others:** calendar (days, Friday and Saturday nights, public holidays) and game mix (share of win from slots, tables and sports betting).
  The calendar comes from Python's built-in calendar, plus a written list of US federal holidays.

- Any calculated or engineered feature should be clearly identified as a derived feature and should not be presented as an original field from the source data.

  What we did: in the input and calendar tables every created column is named derived_…, so it can't be mistaken for a Board figure. The one exception is the main monthly-win table (see above).

- Document the data preparation process so it is clear:

  What we did: documented across several files, listed below. A single data-preparation document isn't written yet; that is a known gap.

- What information came directly from the source reports

  What we did: from the release, the win for the month and the same month last year, for 20 areas. From the big report, win by game and slot type, and venue, table and machine counts and hold. Every field is listed in docs/week4-report-fields.md.

- What information was calculated during preprocessing

  What we did: worked out ourselves:
  – the month each report covers;
  – which figure counts, the newest published (D9);
  – a 'revised' flag and the % revision;
  – the county and market grouping;
  – win converted from thousands to dollars.

- What features were created for model training

  What we did: the 14 model inputs above. Their definitions and checks are in data/features/feature_checks.md.

- What data quality issues were found

  What we did: found and logged 12 issues. The main ones:
  – a mislabelled link;
  – a picture table (January 2025);
  – figures revised after first publication: 23 by more than 1%, all confirmed as genuine Board revisions;
  – a wrong summary page in May to July 2025;
  – two unreadable files;
  – three layout changes;

– a venue count that changed meaning in 2018;
– blank games.

- How missing or inconsistent data was handled

What we did: each issue has a recorded fix:
– the month is read from the link text;
– OCR, then cross-checks;
– the newest figure is used;
– detail pages are used instead of the bad summary;
– blanks recorded as 0 where the totals confirm it;
– 2020 and 2021 left out (D23).

- The final dataset should be suitable for time series forecasting and should preserve the correct chronological order of the data.

What we did: one row per area per month, in date order. All 20 areas are present in all 117 months. The only gap is the deliberate COVID one.

- Do not randomly split the data because future information should not be used to predict past periods.

What we did: no random split anywhere. Every test month is forecast by a model trained only on months published at least two months earlier, using figures as first published.
As a check, rebuilding the baseline from the model-input table gives exactly the same results (5.75% Strip, 7.24% Reno).

-

Step 3: Create a Baseline Model

Start with a simple baseline.

What we did: two simple rules, with no training:
– **'Same month last year':** November 2026 forecast = November 2025's figure.
– **'Last published':** the newest figure out, two months back.

- For example, the next month revenue can initially be predicted using the previous month revenue or a moving average.

What we did: 'previous month' can't be used honestly, because last month's figure isn't published when the forecast is made. So 'last published' (two months back) is the honest version.
A moving-average baseline was not tested. The 3- and 12-month averages are used as model inputs instead. That is a known small gap.

- The baseline is important because every advanced model should perform better than a simple approach.

What we did: every model was scored on the same 43 months as the baselines, and judged against 'same month last year'.

- Do not assume that a complicated model is automatically better.

What we did: proved it. The simplest rule beat every complex model on the Strip's average miss, and tied on Reno. 'Last published' was much worse (10.4%), because it ignores the season: it forecasts December from October and misses the December peak.

Step 4: Train Multiple Forecasting Models

- Train multiple models so their performance can be compared.

What we did: seven models, each rebuilt every test month:
– ARIMA;
– Seasonal ARIMA, and Seasonal ARIMA with calendar inputs, game-mix inputs, or both;
– Random Forest (300 trees);
– XGBoost (200 trees).
Later we added a bottom-up method (D50), now a Strip Challenger. It forecasts slots and tables separately and replaces last year's baccarat luck with a normal hold.
What they are, and the tools:
– **ARIMA:** learns how each month relates to the months just before it.
– **Seasonal ARIMA:** also learns how each month relates to the same month a year earlier. The inputs (calendar, game mix) are added to it.
– Both use statsmodels, a standard Python library for statistical models. Each month, a score called AIC picks the best of four settings.
– **Random Forest:** many decision trees, each a chain of yes/no questions about the inputs; their answers are averaged. Uses scikit-learn, the most common Python machine-learning library.
– **XGBoost:** trees built one after another, each correcting the last. Uses the XGBoost library.
Code: a Python loop (scripts/compare_models.py and tree_models.py) refits every model for each test month and saves its forecast.

- You can select models such as ARIMA, XGBoost, Random Forest, or LSTM.

What we did: ARIMA, XGBoost and Random Forest. LSTM was left out by recorded choice (D42): it's a neural network that needs far more data than the fewer than 120 clean months available, and a large install. The tree models already struggled with this little data.

- At least two models should be compared with the baseline.

What we did: seven models against two baselines.

- The objective is not to use the most complicated model.

What we did: the production model for the Strip is the simplest one tested (D28).

- The objective is to select the most suitable model for the business problem.

What we did: chose on accuracy and on what the business needs: easy to explain, nothing to maintain, and anyone can check it. Where a more complex model earned it, as with Reno, it was promoted through the agreed rule.

Step 5: Use Time Based Backtesting

- Use historical backtesting instead of a random train and test split.

What we did: a month-by-month replay of the past instead of a random split.

- For example, train the model using historical months and predict the next month.

What we did: the same, but two months ahead, because that's what is possible in practice. To forecast September 2026, models are trained on figures to July 2026, the newest the Board had published.

- Then move the training window forward and predict the following month.

What we did: for October 2026, models are retrained with August added, and so on. Training always starts in January 2015 and grows by one month each time.

We found and fixed a bug where the tree models' window didn't grow. The results shown are from the corrected run.

- Continue this across several periods.

What we did: 43 test months, March 2023 to September 2026: 774 forecasts (43 months × 2 markets × 9 methods). The test starts in March 2023 because inputs need 14 months of history after the COVID gap.

- This shows how the model would have performed if it had actually been running during those months.

What we did: inputs use figures as first published, so the test never uses a later correction that wasn't known at the time. Each forecast is scored against the Board's newest figure.

Step 6: Evaluate the Models

Compare each model using suitable forecasting metrics.

What we did: scored every model with all three of the brief's measures, plus bias (whether forecasts lean too low or too high).

- Use MAE.

What we did: MAE is the average size of the miss in dollars. Example: the Strip baseline is off by $43.6m a month on average. It is the measure reported to finance (D31).

- Use RMSE.

What we did: RMSE is like MAE, but big misses count more. Example: the Strip baseline scores $58.8m. It is used as a guard: a new model must not make big misses worse (D31).

- Use MAPE.

What we did: MAPE is the average miss as a % of the actual figure. Example: the Strip baseline scores 5.75%. It is the headline measure, because it compares markets of very different size fairly (D31).

- Also record training time and prediction time.

What we did: recorded seconds per forecast for every model: from instant for the baselines to under 1 second for the slowest ARIMA.

Training and prediction are timed together, not separately. That is a known small gap.

- Create a comparison table.

What we did: filled in below, for both markets, two months ahead over 43 months. Bold is best in each column.

Model MAE RMSE MAPE Training Time

Baseline

ARIMA

XGBoost

LSTM

Model	Strip MAE	Strip RMSE	Strip MAPE	Reno MAE	Reno RMSE	Reno MAPE	Time per forecast
Same month last year (baseline; Strip production)	$43.6m	$58.8m	5.75%	$4.8m	$6.3m	7.24%	instant
Last published (baseline)	$76.7m	$90.9m	10.4%	$7.2m	$8.9m	11.5%	instant
ARIMA	$58.1m	$73.5m	7.8%	$7.0m	$8.2m	10.9%	0.02–0.03 s
Seasonal ARIMA	$49.0m	$61.6m	6.6%	$6.0m	$10.6m	9.5%	0.08–0.13 s
Seasonal ARIMA + calendar	$46.5m	$58.2m	6.3%	$5.2m	$6.6m	8.1%	0.21–0.57 s
Seasonal ARIMA + game mix (Reno production, V2)	$49.9m	$63.9m	6.7%	**$4.7m**	**$6.0m**	**7.21%**	0.37–0.38 s
Seasonal ARIMA + both (Strip Challenger)	$45.7m	$58.4m	6.1%	$5.0m	$6.4m	7.8%	0.66–0.85 s
Random Forest	$50.0m	$65.7m	6.7%	$4.9m	$6.1m	7.5%	0.19 s
XGBoost	$52.3m	$70.3m	7.1%	$5.2m	$6.4m	7.9%	0.16 s
Bottom-up, baccarat luck out (later test; Strip Challenger)	**$40.2m**	**$54.6m**	**5.28%**	–	–	–	under 0.01 s
LSTM	not tried (D42)						

Reading the table: on the Strip, 'same month last year' beat every model in the original comparison. The later bottom-up method beats it but leans further too low, so it runs as a Challenger rather than replacing it. On Reno, Seasonal ARIMA + game mix and the baseline were nearly level over 43 months. ARIMA was clearly better over the last 12 months, and was promoted under the rule.
Times are seconds per forecast, training and prediction together.

The student should explain which model should be selected and why.

What we did: 'same month last year' was selected for production (D28). It has the lowest average miss on the Strip in % and dollars, ties on Reno, has nothing to train or maintain, and anyone can check it.

Reno later moved to Seasonal ARIMA + game mix, once it consistently won on recent months under the promotion rule.

The decision should not be based only on one metric.

What we did: looked at all three measures. They disagree a little: on the Strip, two ARIMA models have slightly smaller big misses (RMSE $58.2m against $58.8m) but a larger average miss. That is exactly why we agreed which measure leads (MAPE) and which act as guards (RMSE, bias) (D31).

Model complexity, maintenance, prediction speed, and business usefulness should also be considered.

What we did: compared on all four:

– **Complexity:** the baseline is one look-up; the others are hundreds of trees or a statistical model.

– **Maintenance:** the baseline has none; the others need monthly retraining plus extra data from the 48-page report.

– **Speed:** all are under 1 second.

– **Usefulness:** the baseline is easy to trust and check; the tree models are hard to explain and no more accurate.

Step 7: MLflow Experiment Tracking

Use MLflow to track all experiments.

What MLflow is: a free, widely used tool for recording machine-learning experiments. Each 'run' keeps a model's settings, scores, charts and the fitted model together, and runs can be searched and compared side by side in a web page.

Ours keeps everything in a small database file in the project (mlflow/mlflow.db). It's viewed in the browser at localhost:5050, and nothing goes online. scripts/mlflow_log.py records the runs.

What we did: MLflow 3.18 on this computer, with usage statistics switched off. One experiment holds 20 runs: 9 methods × 2 markets, plus the two bottom-up runs. Each is named like 'Reno | SARIMA + game mix'.

For every experiment, record the model name.

What we did: in the run name, plus 'model' and 'market' tags, so runs can be filtered and compared.

Record the training period.

What we did: two fields per run. Training period: January 2015 to two months before each forecast, without 2020–21. Test period: March 2023 to September 2026.

Record the features used.

What we did: a 'features' field per run, for example 'slot share of win (two months back); sports pool share of win (two months back)'.

Record the hyperparameters.

What we did: recorded per run. Examples: the ARIMA version chosen each month; Random Forest 300 trees; XGBoost 200 trees, depth 3, learning rate 0.05.

Record MAE.

What we did: MAE_m on every run.

Record RMSE.

What we did: RMSE_m on every run.

Record MAPE.

What we did: MAPE_pct on every run, plus last-12-month MAPE, bias, and the miss for each of the 43 months, so the error can be charted over time.

Record training time.

What we did: seconds per forecast on every run. As noted under Step 6, this covers training and prediction together.

Record model artifacts.

What we did: every run stores its fitted model and its 43 forecasts. The production models are saved in MLflow's standard model format ('pyfunc'), so any of them can be loaded by name and asked for a forecast.

Record forecast charts.

What we did: every run has a chart of actual against forecast over the 43 months, with the MAPE in the title.

The purpose is to have a proper experiment history instead of keeping different versions in separate notebooks.

What we did: every experiment is in one place, side by side, and can be rebuilt from its script (scripts/mlflow_log.py).

Step 8: Model Selection

After comparing the experiments, select the best model.

What we did: selected 'same month last year', approved on 12 November 2026 (D28).

The student should document why that model was selected.

What we did: docs/week4-model-comparison.md is the short decision document. The reasons are also set out in the decision log (D26 to D28).

For example, one model may have slightly better RMSE but may be much more difficult to maintain.

What we did: this was exactly our case. On the Strip, Seasonal ARIMA + calendar has a slightly better RMSE ($58.2m against $58.8m), but it needs monthly retraining and extra data, and it is worse on the average miss. So the baseline was kept.

Another model may have slightly lower accuracy but may be easier to explain and deploy.

What we did: in our case the easiest model to explain was also the most accurate on average, so nothing had to be traded off for the Strip.

The final decision should consider both technical and business factors.

What we did: both were weighed. On the technical side: three measures and speed. On the business side: how easy it is to explain, what it costs to maintain, and whether it can be trusted and checked (see Step 6).

Step 9: MLflow Model Registry

Register the selected model in MLflow.

What the registry is: MLflow's list of official model versions. Each version is a saved model. A label (MLflow calls it an alias) marks which version is 'production' and which is 'challenger'. So monitoring can ask for 'the production model for Reno' without needing to know which version that is.

What we did: one registered model per market: 'Revenue Forecast Model – LV Strip' and 'Revenue Forecast Model – Reno'.

Create a production model version.

What we did: a 'production' label marks the version in use: Strip version 1 and Reno version 2. A 'challenger' label marks the one running alongside.

For example:

Revenue Forecast Model V1

What we did: V1 = 'same month last year', for both markets.

Later, if an improved model is trained, create:

Revenue Forecast Model V2

What we did: Reno V2 = Seasonal ARIMA + game mix, in production from the November 2026 forecast.

Document why the new version should replace the old version.

What we did: Reno V2's registry entry records why. It passed the promotion rule three months in a row (last-12-month miss 5.6% against 6.8%, lower RMSE and bias), and the Product Owner approved it (D36) after an evidence pack (docs/week4-reno-switch-evidence-pack.md).

The student should maintain a clear model history.

What we did: every version carries its decision number and reason, and the decision log tells the full story (D28, D32, D33, D36, D45).

Step 10: Champion and Challenger Model

Use the production model as the Champion model.

What we did: the Champion is the production model for each market: Strip V1 ('same month last year') and Reno V2 (Seasonal ARIMA + game mix).

Train another model as the Challenger.

What we did:
– **Strip:** two Challengers (D51). Seasonal ARIMA + both picks up growth; bottom-up takes out baccarat luck.
– **Reno:** the old Champion, 'same month last year', became the Challenger when V2 was promoted. That is the way back (D33).

Both models should make predictions on the same new data.

What we did: every monthly run forecasts with the Champion and every Challenger for the same month. November 2026, Strip: production $784.3m, Seasonal ARIMA + both $776.6m, bottom-up $796.8m.

Compare their performance over several periods.

What we did: compared year by year, over the last 12 months, and over all 43 months.
Example: Reno's Challenger was behind in 2023 and 2025 but well ahead in 2026 (5.8% against 8.1%). That is why the rule looks at both recent and long-run results.

The Challenger should only replace the Champion if it consistently performs better according to the agreed evaluation criteria.

What we did: monitoring runs the agreed check every month. Reno's Challenger passed it four months in a row and was promoted after approval. The Strip's Challengers don't pass now: bottom-up is blocked because its forecasts lean too low.

The student should define the rules for model promotion.

What we did: the rule (D32), per market: at three monthly checks in a row, the Challenger is at least 0.5 points better over 12 months, no more than 0.25 points worse over the full test, and no worse on big misses or lean. The Product Owner approves (D34).
– **Way back (D33):** the old Champion becomes the Challenger.
– **Two Challengers (D51):** if both qualify at once, the lower 12-month miss wins.
– **Why these settings:** we tested five versions. A simple 'better over 12 months' rule would have switched 8 or 9 times on noise.

Step 11: Model Observability

Once the model is considered deployed, monitor its performance over time.

What we did: a monthly monitoring run (scripts/monitor.py, D45). It checks the new data, scores the forecast made two months earlier, updates the measures and alerts, runs the Champion/Challenger check, and makes the next forecast with its range.
It was replayed over all 43 test months, and run live on September 2026. It is run by hand on the first working day of each month, following a written routine (D54).
How it's built: a Python script that reads the clean tables and saved forecasts, and writes plain files any tool can open:
– monthly_status.csv: one row per market per month;
– forecast_log.csv: every forecast, with its model, version and range;
– thresholds.json: the alert levels;
– report_<month>.md: a readable report for each month.

Track actual revenue.

What we did: the actual figure each month, in monthly_status.csv.

Track forecast revenue.

What we did: the forecast that was made for each month, and every new forecast with its model, version and 80% range (forecast_log.csv).

Track prediction error.

What we did: the miss each month as a signed % (+ means actual above forecast), and whether the actual landed inside the 80% range.

Track MAE over time.

What we did: rolling 12-month MAE, recalculated every month, alongside RMSE and bias.

Track MAPE over time.

What we did: rolling 12-month MAPE every month, charted on the dashboard against its alert line.

Track missing input data.

What we did: a data-check column records any check that fails. A late Board release raises an alert 7 days after it's due (D38).

Track model version.

What we did: the version for each market is stamped on every forecast and status row, and matches the MLflow registry.

Track last retraining date.

What we did: recorded on every forecast. Models that learn settings show the month they were retrained to (September 2026). 'Same month last year' shows 'not applicable', because there is nothing to retrain.

Track model health.

What we did: each market gets a status every month: **Healthy**, **Investigate** (an alert fired) or **Data issue** (the new data failed a check).

The purpose is to identify when the production model starts performing worse.

What we did: proven on the replay. It would have flagged the Strip in 5 months (capacity changes in December 2023 and January 2024, a slow decline in August 2024, a bad patch in February–March 2025) and Reno in 8 months.

Step 12: Data Quality Monitoring

Before generating a forecast, validate the incoming data.

What we did: the data check is the first step of every monitoring run. A month is used only if every check passes (D38).

Check whether the required month exists.

What we did: the month must appear exactly once in the downloads. If the Board is late, an alert fires 7 days after the due date.

Check whether the revenue value is valid.

What we did: no zero or negative values. Areas must add up to their county, and counties to the state, within $10. The two independent readers must agree on every number.

Check whether duplicate records exist.

What we did: each month must appear exactly once, with exactly 20 areas.

Check whether expected markets are available.

What we did: all 20 areas must be present, plus the game-mix data for models that use it.

Check whether the data structure has changed.

What we did: a changed layout breaks the 20-area and add-up checks, so it is caught. All known layouts are catalogued.

The model should not automatically generate a forecast when the input data is incomplete or incorrect.

What we did: a month that fails is not used, and the market is marked Data issue (D38). 'Same month last year' can keep forecasting, because it needs only last year's checked figure, and it is labelled as such. More complex models fall back to it.

Tested on a deliberately broken copy of September 2026: it behaved exactly so.

Step 13: Model Drift

Monitor whether the behaviour of the incoming data changes compared with the data used to train the model.

What we did: data drift is watched through the capacity alert: tables or slot machines changing by more than 5% on a year earlier. It fired for the Strip in December 2023, January 2024 and February 2025, and for Reno in May 2025 and May 2026.

Also monitor whether model performance becomes worse over time.

What we did: performance drift is watched through two alerts.
– **Fast:** 2 of the last 3 months among the model's worst 10% of misses.
– **Slow:** the 12-month average miss more than 1.5 points above the model's normal.
– **Range:** 7 or fewer of the last 12 actuals inside the 80% range (D53).

The student should understand the difference between data drift and performance drift.

What we did: they are monitored separately.
– **Data drift:** the world the model sees changes, such as fewer machines or a closure.
– **Performance drift:** the forecasts themselves get worse.
One can happen without the other.

A single unusual month should not automatically trigger retraining.

What we did: built in. The fast alert needs 2 bad months in 3, and any alert leads to an investigation, not automatic retraining. Example: Reno's 35.5% miss in January 2024 raised no alert on its own.

The student should define when drift becomes important enough to require investigation.

What we did: the drift rule (D37). Any alert turns the status to Investigate, and the analyst explains the cause within the month. Thresholds come from each model's own test history, not guesses. Bias and hold were tested as alerts and were too noisy, so they are shown but not alerted.

Step 14: Retraining Strategy

Create a retraining policy.

What we did: a written retraining policy (D39), with two layers: routine monthly retraining, and a full model review yearly or on certain alerts.

The student should decide when retraining is required.

What we did:
– **Routine:** every month, for models that learn settings.

– **Full review:** once a year, plus when a slow alert fires, a capacity change is real (an opening or closure), or a fast alert has a lasting cause.

A review re-runs the whole comparison. Any change still goes through the promotion rule and the Product Owner.

For example, retraining may be considered when prediction error remains above the accepted threshold for multiple periods.

What we did: that is our slow alert: the 12-month average miss more than 1.5 points above normal triggers a full review.

Retraining may also be considered when the data pattern changes significantly.

What we did: that is our capacity alert: tables or machines changing by more than 5% for a real reason triggers a review. Example: the Mirage and Tropicana closures in 2024.

The exact rule should be based on project results rather than an arbitrary value.

What we did: tested on our 43 months.

– **Strip ARIMA model:** monthly retraining 6.14%, yearly 6.52%, never 7.67%. Retraining clearly helps.

– **Reno:** near-neutral.

Alert thresholds come from each model's own error history.

Step 15: Monitoring Dashboard

Create a dashboard for model monitoring.

What we did: a dashboard (dashboard/app.py), on this computer only (D44, D47). One page with a market picker, checked in the browser for both markets.

The tools: Streamlit, a Python library that turns a script into an interactive web page, and Altair for the charts. It reads the monitoring files, so it always shows the latest run. It opens in the browser at localhost:8504.

The dashboard should show actual revenue.

What we did: the 'Actual against forecast' chart.

It should show forecast revenue.

What we did: on the same chart, with the 80% range shaded. The next forecast and its range are shown for the production model and each Challenger.

It should show forecast error.

What we did: bars showing the monthly miss, with bad months in red.

It should show historical revenue trends.

What we did: 'Historical revenue, 2015 to now', with the 2020–21 gap shown as a gap.

It should show the current production model.

What we did: 'In production for the next forecast', naming the model and its Challengers.

It should show the model version.

What we did: 'Production version': V1 for the Strip, V2 for Reno.

It should show the latest evaluation metrics.

What we did: the 12-month MAPE, MAE, RMSE and bias for the model in use, with a chart of MAPE over time against its alert line.

It should show drift status.

What we did: a drift section with the status this month, months flagged in the last 12, months inside the 80% range, the promotion check, and a table of every flagged month with its alerts.

It should show the last retraining date.

What we did: 'Last retrained'. For Reno V2 it shows September 2026; for the Strip's rule it shows 'n/a (a rule)'.

It should show whether the model is healthy or requires investigation.

What we did: a coloured status badge next to the market name: green Healthy, amber Investigate, red Data issue.

Step 16: Product Decisions

Document the important product decisions.

What we did: a decision log of 54 entries (D1 to D54). Each records what was decided, why, with the evidence, and who decided.

Explain which model should go into production.

What we did: Strip: 'same month last year' (D28). Reno: Seasonal ARIMA + game mix (V2), promoted after it passed the rule and was approved (D36).

Explain which evaluation metric matters most.

What we did: MAPE (the average % miss) is the headline. It decides promotion, drift and whether an error is acceptable. RMSE is a guard against bigger big misses. Bias is tracked, because Reno's forecasts were too low in 28 of 43 months, which only bias shows. MAE is reported in dollars for finance (D31).

Explain when a model should be retrained.

What we did: routine retraining every month. A full review yearly, and on a slow alert, a real capacity change, or a fast alert with a lasting cause (D39).

Explain when a Challenger should replace the Champion.

What we did: only when it passes the promotion rule three months in a row and the Product Owner approves (D32, D34). The old Champion stays on as the Challenger, so the change can be reversed (D33).

Explain what should happen when incoming data is incomplete.

What we did: the month is not used, the market is marked Data issue, and the analyst is alerted. Forecasts continue from 'same month last year', labelled. A repair is allowed only if every check then passes (D38).

Explain what should happen if a new model performs worse than the existing model.

What we did: it is not promoted. It keeps running as a Challenger if it is close, or is dropped at the yearly review (D40). If a promoted model later gets worse, the way back returns the old one, and operations can fall back to 'same month last year' at once in an emergency (D34).

Explain who should approve replacing the production model.

What we did: the **Product Owner** approves, after the analyst presents an evidence pack: the four rule checks, what changes, the history and the way back. Finance is consulted. The person proposing never approves alone (D34). We used this process for the Reno switch.

Step 17: Risks

Document the main risks.

What we did: an approved risk register (docs/week4-risk-register.md, D52). It has the brief's seven risks plus five of our own: baccarat luck, a low lean, report layout changes, revisions, and a manual monthly run.
Each risk has what we saw, the protection, likelihood, impact, what's left, and an owner.

The model may perform well during training but poorly on future data.

What we did: seen here: the tree models fitted their training data well but lost to the baseline on unseen months. **Protection:** every model is judged only on months it never saw. **Left over:** Low.

Historical patterns may change.

What we did: seen here: COVID, stalled growth, 20% fewer Strip machines, a stronger December. **Protection:** slow and capacity alerts, monthly retraining and reviews. **Left over:** Medium, because an alert says something changed, but a person must find out why.

Seasonal behaviour may affect performance.

What we did: seen here: the December peak, and low February and August. **Protection:** the production model carries the season by design, and Seasonal ARIMA models it. **Left over:** Low.

Missing monthly data may produce incorrect forecasts.

What we did: seen here: a picture table, unreadable 2025 pages, and a four-week publishing delay. **Protection:** two-months-ahead forecasts, the bad-data rule and fallback, and a late alert. **Left over:** Low.

A new model may perform better on one period but worse overall.

What we did: seen here: Reno's Challenger met a simple rule in October 2024, then did worse through 2025. **Protection:** the promotion rule needs a margin, a full-test check and three months in a row, plus the way back. **Left over:** Low.

The business may trust forecasts without reviewing uncertainty.

What we did: seen here: forecasts were single numbers. **Protection:** every forecast now has an 80% range (Strip November 2026: $724m to $893m), and an alert fires if too few actuals land inside (D53). **Left over:** Low, down from High.

The model may remain in production even after its performance has degraded.

What we did: not seen on the production models so far. **Protection:** monthly monitoring with health status, alerts, the way back and an emergency fallback. **Left over:** Low, provided the monthly run happens. It is manual, with a written routine (D54).

Step 18: Final Deliverables

The final project should contain a clean historical casino revenue dataset.

What we did: data/clean/: monthly win for 20 areas × 117 months, win by game type, and capacity and hold. All read two ways and cross-checked.

It should contain exploratory analysis.

What we did: docs/week4-exploratory-analysis.md (and Word copy): 2015 onward, four markets, seasons, capacity and hold, four charts (D48). The charts are drawn with pandas and matplotlib, Python's standard table and charting libraries (scripts/eda_charts.py).

It should contain a baseline forecasting model.

What we did: two baselines. 'Same month last year' is the Strip's production model.

It should contain at least two additional forecasting models.

What we did: seven models, plus the later bottom-up method.

It should contain time based backtesting.

What we did: 43 months, 774 forecasts, two months ahead, with no future information used.

It should contain model evaluation results.

What we did: the comparison table in Step 6 (data/model/model_comparison.md).

It should contain MLflow experiment tracking.

What we did: 20 MLflow runs with settings, measures, charts and models.

It should contain a registered production model.

What we did: MLflow registry: Strip V1 and Reno V2 labelled production.

It should contain Champion and Challenger comparison.

What we did: monthly in monitoring, with the agreed rule. Reno V2 was promoted through it.

It should contain data quality monitoring.

What we did: checks before every forecast, tested on a broken month.

It should contain model drift monitoring.

What we did: data drift (capacity) and performance drift (fast, slow and range alerts), every month.

It should contain a retraining strategy.

What we did: a written policy, chosen from tests on our results (D39).

It should contain a model monitoring dashboard.

What we did: Streamlit, both markets, on this computer.

It should contain a GitHub repository.

What we did: github.com/tezla-timemachine/casino-revenue-forecasting (public, D49). It has the scripts, clean data, results, monitoring, dashboard and documents. The PDFs and MLflow store are rebuilt by scripts.
Uploaded with git, which records every change as a numbered version, and GitHub's command-line tool (gh).

It should contain a short document explaining why the final production model was selected.

What we did: docs/week4-model-comparison.md: why 'same month last year' was chosen, the results behind it, and the later bottom-up test. The Reno switch has its own evidence pack.